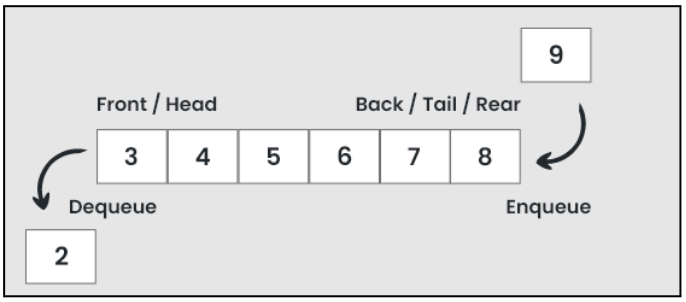
Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Unit 4 - Queues

Introduction

- A **Queue** is a fundamental data structure in computer science that is used to store and manage data in a specific order.
- It is a ***linear data structure*** that follows the **FIFO (First In, First Out) principle**, which means the first element inserted into the queue will be the first one to be removed.
- A real-life example of a queue is a line of people waiting at a ticket counter – the person who comes first is the first to be served.



FIFO Principle in Queue

- The **FIFO principle** ensures fairness in processing elements: the first element added will be the first element processed or removed.
- Just like "**First Come, First Serve**" in real life, a queue ensures that no element is skipped or delayed unfairly.

Basic Terminologies of Queue

- **Front (Head):** The position of the first element in the queue, which will be removed during a dequeue operation.
- **Rear (Tail):** The position of the most recently added element in the queue.
- **Size:** The current number of elements present in the queue.
- **Capacity:** The maximum number of elements the queue can hold.

Queue Operations

1. **Enqueue:** Adds an element at the rear end of the queue. If the queue is already full, an *overflow error* occurs.
2. **Dequeue:** Removes an element from the front of the queue. If the queue is empty, an *underflow error* occurs.
3. **Peek / Front:** Returns the element at the front of the queue without removing it.
4. **Size:** Returns the number of elements currently in the queue.
5. **isEmpty:** Returns true if the queue has no elements, otherwise false.
6. **isFull:** Returns true if the queue has reached its maximum capacity, otherwise false.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Types of Queues

1. **Simple Queue (Linear Queue):**
   - Follows the standard FIFO principle.
   - Insertion is done at the rear end, and deletion is done at the front end.
   - Can be implemented using arrays or linked lists.
2. **Circular Queue:**
   - A more efficient version of the simple queue where the last position connects back to the first.
   - Helps in utilizing unused memory space in a simple queue.
3. **Double-Ended Queue (Deque):**
   - A queue where insertion and deletion can be done from both ends.
   - Types of Deque:
     i. **Input Restricted Deque:** Insertion is allowed only at one end, but deletion can be done from both ends.
     ii. **Output Restricted Deque:** Deletion is allowed only from one end, but insertion can be done at both ends.
4. **Priority Queue:**
   - A special type of queue where each element is assigned a priority.
   - Elements with higher priority are served before those with lower priority.
   - Two types:
     i. **Ascending Priority Queue:** The element with the smallest priority value is removed first.
     ii. **Descending Priority Queue:** The element with the largest priority value is removed first.

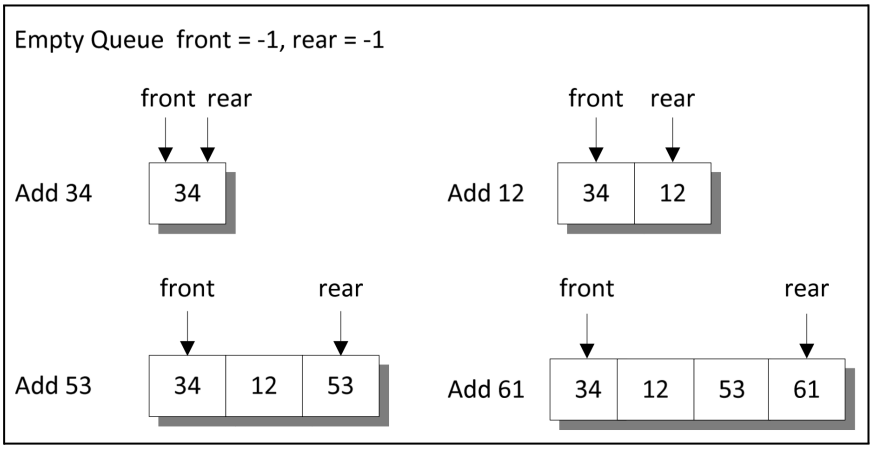
Applications of Queue

1. **Buffer Management:** Queues act as buffers in cases of speed mismatch between producer and consumer devices.
   Example: The keyboard produces keystrokes at a slower rate, while the CPU processes data faster. The queue helps balance this mismatch.
2. **Resource Management:** Queues are used when multiple consumers share a single resource.
   Example: Multiple processes waiting to use a single CPU.
3. **Computer Networks:** Used in routers, switches, and mail queues to manage data packets waiting for transmission.
4. **Operating Systems:** Widely used in algorithms such as:
   - CPU Scheduling (Round Robin)
   - Disk Scheduling
   - Memory Management
5. **Algorithm Design:**
   - Breadth First Search (BFS) in Graphs, Level Order Traversal in Trees
   - Handling asynchronous tasks like print spooling and IO operations

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Array implementation of Queue

1. A Queue can be implemented using an array, where we maintain two variables:
    a. **Front** → points to the first element of the queue.
    b. **Rear** → points to the last element of the queue.
2. Initially, both Front and Rear are set to -1 (indicating an empty queue).
3. Insertion of elements (Enqueue) is done at the rear end, and deletion (Dequeue) is done from the front end.



Steps in Array Implementation

Initialize

- front = -1
- rear = -1
- queue[MAX] → where MAX is the maximum size of the array.

Enqueue (Insert element)

- Check ***overflow***: If rear == MAX - 1, the queue is full.
- If the queue is empty (front == -1), set front = 0.
- Increment rear by 1 and insert the new element at queue[rear].

Algorithm Enqueue(item)

1. If rear == MAX - 1:
   Print "Queue Overflow"
   Exit
2. If front == -1:
   front ← 0
3. rear ← rear + 1
4. queue[rear] ← item

Dequeue (Delete element)

- Check ***underflow***: If front == -1 or front > rear, the queue is empty.
- Remove the element at queue[front].

- Increment front by 1.
- If after deletion, front > rear, reset both front = rear = -1.

Algorithm Dequeue()

1. If front == -1 or front > rear:
   Print "Queue Underflow"
   Exit
2. item ← queue[front]
3. front ← front + 1
4. If front > rear:
   front ← -1
   rear ← -1
5. Return item

Peek (View front element)

If the queue is not empty, return the element at queue[front].

isEmpty

Returns true if front == -1.

isFull

Returns true if rear == MAX - 1.

Circular Array Implementation of Queue

Need for Circular Queue

1. In a simple linear queue using arrays, once elements are dequeued, the freed spaces cannot be reused.
   - Example: For queue[5], if we enqueue 5 elements, and then dequeue 3, only 2 elements remain.
   - But we cannot insert new elements even though space exists at the beginning, because rear is stuck at the last index. This leads to wastage of memory.
2. To solve this, we use a Circular Queue, where the last position is connected back to the first position.
   - This makes the queue a circular buffer where unused space at the beginning can be reused.

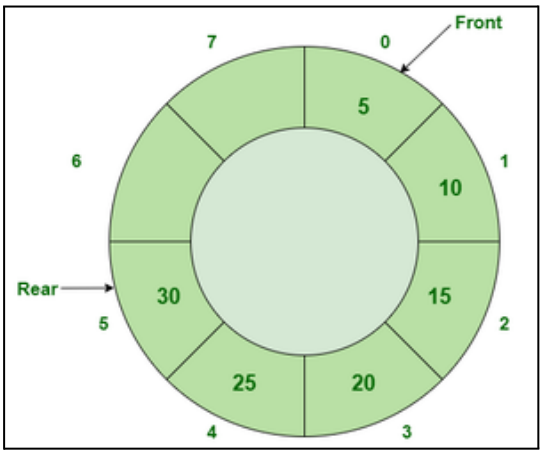
In a circular queue, the positions are arranged in a circular manner, so when the rear reaches the last index (MAX - 1) and there is free space at the beginning, it wraps around to index 0, and in the same way, the front pointer also moves circularly when elements are dequeued.

Representation

- **Front:** Index of the first element in the queue.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

- **Rear:** Index of the last element in the queue.
- **Empty Condition:** front == -1.
- **Full Condition:** (rear + 1) % MAX == front.



Enqueue (Insert)

Algorithm Enqueue(item)

1. If ((rear + 1) % MAX == front):
   Print "Queue Overflow"
   Exit
2. If (front == -1): // Queue is empty
   front ← 0
   rear ← 0
   Else
   rear ← (rear + 1) % MAX
3. queue[rear] ← item

Dequeue (Delete)

Algorithm Dequeue()

1. If (front == -1):
   Print "Queue Underflow"
   Exit
2. item ← queue[front]
3. If (front == rear): // Only one element was present
   front ← -1
   rear ← -1
   Else
   front ← (front + 1) % MAX
4. Return item

Peek

If front == -1:

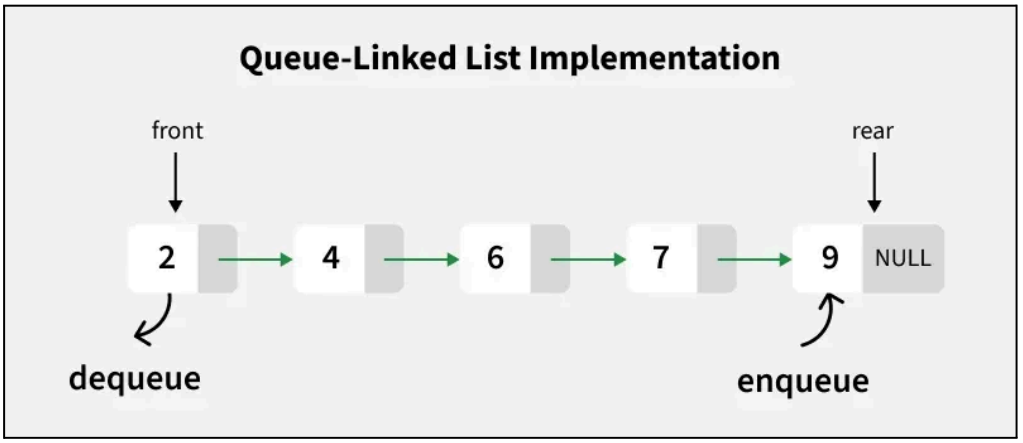
```
    Print "Queue Empty"
Else
    Return queue[front]
```

Advantages of Circular Queue

- Efficient utilization of memory (no wasted space like linear queue).
- Useful in buffer management (I/O buffers, circular buffers in OS, and network traffic management).

Linked List Implementation of Queue

- In an array implementation, the size of the queue is fixed. If the array becomes full, we cannot insert more elements even if memory is available elsewhere.
- In a linked list implementation, the queue can grow and shrink dynamically as per memory availability.
- This avoids the problem of overflow (except when memory is exhausted).



In this implementation, each element of the queue is represented as a node in a linked list. Each node has two parts:

- **Data:** The actual value stored in the queue.
- **Next:** Pointer to the next node.

Two pointers are maintained:

- **Front:** Points to the first node (head) of the queue, which will be deleted first during dequeue.
- **Rear:** Points to the last node (tail) of the queue, where new nodes are inserted during enqueue.

Operations

Enqueue (Insert)

- Create a new node with given data.
- If the queue is empty (front == NULL), set both front and rear to the new node.

- Otherwise, insert the new node at the end and update the rear.

Algorithm Enqueue(item)

1. Create newNode with data = item and next = NULL
2. If front == NULL:
   front ← newNode
   rear ← newNode
   Else:
   rear.next ← newNode
   rear ← newNode

Dequeue (Delete)

- If the queue is empty (front == NULL), display an underflow message.
- Otherwise, remove the node at front, move front to the next node, and free the deleted node.
- If after deletion front == NULL, also set rear = NULL.

Algorithm Dequeue()

1. If front == NULL:
   Print "Queue Underflow"
   Exit
2. temp ← front
3. item ← temp.data
4. front ← front.next
5. If front == NULL:
   rear ← NULL
6. Free(temp)
7. Return item

Peek (Front Element)

- If the queue is not empty, return the data at front.

isEmpty

Returns true if front == NULL.

Advantages of Linked List Queue

- Dynamic size (no wastage of memory like arrays).
- No fixed limit except system memory.

Disadvantages

- Requires extra memory for storing pointers.
- Slightly slower than array implementation due to dynamic memory allocation.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Circular Linked List Implementation of Queue

- A circular linked list is a variation of a linked list where the last node points back to the first node, forming a circle.
- In a circular queue using a linked list:
  - We maintain only a rear pointer.
  - The rear->next always points to the front node (first element of the queue).
- This structure makes insertion and deletion efficient because we don't need to traverse the entire list.

Operations

Enqueue (Insert)

- Create a new node with the given data.
- If the queue is empty:
  - Set the new node's next to itself.
  - Set rear = newNode.

  Otherwise:
  - Set newNode->next = rear->next (link new node to front).
  - Set rear->next = newNode.
  - Update rear = newNode.

Dequeue (Delete)

- If the queue is empty (rear == NULL), underflow condition.
- If only one node is present (rear->next == rear):
  - Delete that node and set rear = NULL.

  Otherwise:
  - Let front = rear->next.
  - Set rear->next = front->next.
  - Delete front.

Peek (Front Element)

If the queue is not empty, the front element is rear->next->data.

isEmpty

Returns true if rear == NULL.

Advantages

- Dynamic size (like linked list).
- Efficient circular behavior (like circular array).
- No unused memory positions.

Disadvantages

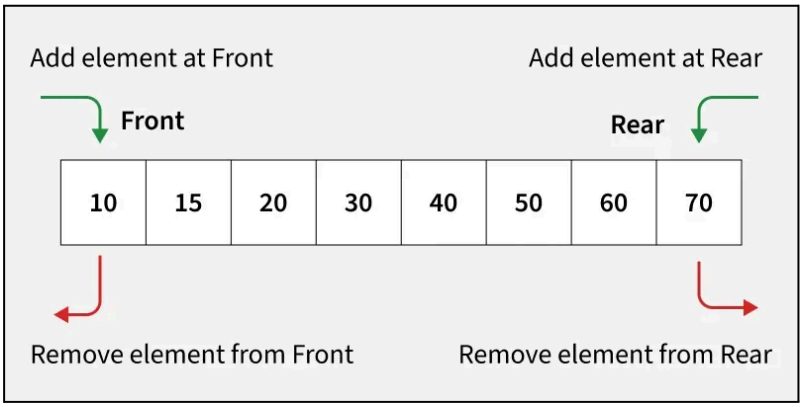
- Requires extra memory for pointers.
- Slightly more complex implementation.

Deque (Double-Ended Queue)

A **Deque** (*pronounced* "***deck***") is a type of queue in which insertion and deletion of elements can be performed at both ends — the front as well as the rear. It is more flexible than a simple queue (insertion only at rear, deletion only at front).

Characteristics

- Deque stands for Double Ended Queue.
- It can be implemented using arrays, linked lists, or circular arrays.
- It allows operations similar to both stacks and queues, depending on usage.



Deque Operations

- **insertFront(x):** Inserts an element at the front of the deque.
- **insertRear(x):** Inserts an element at the rear of the deque.
- **deleteFront():** Deletes an element from the front.
- **deleteRear():** Deletes an element from the rear.
- **getFront():** Returns the front element without removing it.
- **getRear():** Returns the rear element without removing it.
- **isEmpty():** Checks whether the deque is empty.
- **isFull():** Checks whether the deque is full (if implemented using an array).

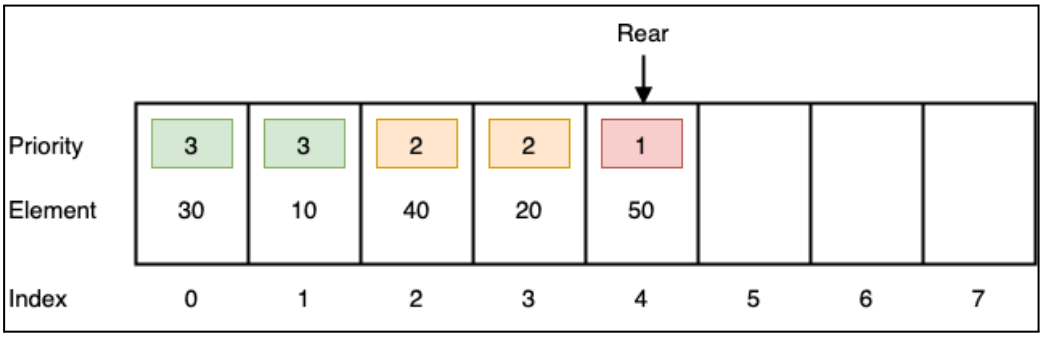
Applications of Deque

1. **Job Scheduling:** Used in operating systems to add or remove jobs from both ends for faster scheduling.
2. **Palindrome Checking:** Characters can be checked from front and rear to test if a word is a palindrome.

3. **Sliding Window Problems:** Helps to quickly find maximum or minimum values in every group of k elements in an array.
4. **Undo/Redo in Text Editors:** Stores recent operations so we can undo or redo actions easily.
5. **Implementation of Stack and Queue:** Since deque allows insertion and deletion at both ends, it can work as both a stack and a queue.

Priority Queue

- A priority queue is a special type of queue in which each element is associated with a priority, and elements are served according to their priority rather than the order in which they arrive.
- In a normal queue (FIFO structure), the first element inserted is the first one to be removed, but in a priority queue, the element with the highest priority is removed first, regardless of its position in the queue.
- If two elements have the same priority, then they are served according to their order of arrival (i.e., FIFO is maintained among elements of equal priority).
- A priority queue can be thought of as a data structure that combines the properties of both queues and sorting, since it always selects the next element based on priority.
- Priority queues are widely used in operating systems, scheduling algorithms, graph algorithms), and network systems where tasks must be processed based on importance.



Types of Priority Queue

Ascending Priority Queue

- In this type, the element with the lowest priority value is dequeued (removed) first.
- For example, in a hospital emergency system, the patient with the most critical condition (lowest health score) is treated first.

Descending Priority Queue

- In this type, the element with the highest priority value is dequeued first.
- For example, in a job scheduling system, the process with the highest importance or weight is executed first.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Implementation of Priority Queue

Using Arrays

- Elements can be stored along with their priority in a simple array.
- While inserting, elements are placed at the end, and while deleting, the array is scanned to find the highest-priority element.
- This method is simple but inefficient, since deletion requires O(n) time.

Using Linked List

- Elements are inserted into a linked list in such a way that the list remains sorted by priority.
- Deletion of the highest-priority element is efficient (O(1)), but insertion takes O(n) time to maintain the order.

Using Heap (Binary Heap)

- A heap is the most efficient way to implement a priority queue.
- Insertion and deletion both can be performed in O(log n) time.
- A max-heap is used when the highest value has the highest priority, while a min-heap is used when the lowest value has the highest priority.

Real-World Applications of Priority Queue

- **Operating System Process Scheduling:** CPU scheduling often uses priority queues where high-priority processes are given preference over low-priority processes.
- **Printer Queue Management:** Printing tasks are arranged by priority so that urgent tasks are printed first.
- **Network Routing and Bandwidth Management:** Data packets with higher priority (like video calls) are transmitted before less important packets (like background downloads).

Lab Assignments

1. Write a program to implement a queue using arrays with enqueue, dequeue, and display operations.
2. Write a program to implement a queue using a linked list with enqueue, dequeue, and display operations.
3. Write a program to implement a double-ended queue (deque) with insert and delete operations at both ends.
4. Write a program to implement a priority queue using an array or linked list with insertion and deletion operations.